

RobomationLAB Coding Guide

Coding-Guide

This document explains RobomationLAB, a robot coding education platform that is part of Robomation's "Execution Engine and Integrated Authoring Environment for Robot Motion Streaming Services".

It provides a detailed explanation in the following order:

- Main features of RobomationLAB - Robot programming methods of RobomationLAB - Principles of robot software modeling - Basic grammar system of RobomationLAB

Table of Contents

1. RobomationLAB - Robot Coding Education Platform
 - 1-1) Robot Coding Program
 - 1-2) Main Features of the Coding Program
 - 1-3) Real-Time Robot Control Method
2. RobomationLAB - Robot Programming Method
 - 2-1) Sequential Execution and Parallel Execution
 - 2-2) setup Function
 - 2-3) loop Function
3. Robot Software Modeling
 - 3-1) Modeling Process
 - 3-2) Roboid Class Definition
 - 3-3) Unique Identifier, urn
4. RobomationLAB - Basic Grammar System for Robot Coding
 - 4-1) Robot Device Objects
 - 4-2) d Function
 - 4-3) c Function
 - 4-4) e Function
 - 4-5) w Function

RobomationLAB - Robot Coding Education Platform

RobomationLAB is a platform optimized for **AI robotics software education**, providing a Chrome web browser-based integrated authoring environment for robot coding education targeting elementary and middle school students.

RobomationLAB offers various robot coding education programs such as **block coding** and **script coding**.

In addition to learning coding theory, learners can actually move and control robots using Robomation's products, providing opportunities to learn both coding and robotics at the same time.

Robot Coding Program

Block Composer

Block Composer is a tool that allows users to easily and quickly control Robomation robots through block coding and learn the basics of robot control.

- An authoring environment optimized for physical computing
- Easy coding for beginners using block Drag & Drop
- Learning environment that provides basic concepts and prevents syntax errors
- Automatic conversion to JavaScript and Python script code
- A collection of predefined blocks with robot-specific functions and various hands-on examples
- Real-time result verification through code execution
- Improvement of problem-solving skills and creativity through block combinations
- Optimized feedback provided through AI-based script code analysis

Script Composer

Script Composer is a tool that allows users to easily and quickly control Robomation robots through script coding and learn the basics of Python, JavaScript syntax, and robot coding.

- Simultaneous support for Python and JavaScript editors
- Automatic code completion and code insertion features by language
- Various hands-on example codes provided for each robot
- Real-time result verification through code execution
- Optimized feedback provided through AI-based script code analysis

Main Features of the Coding Program

The main features of the robot coding programs provided by RobomationLAB are as follows.

1. Chrome web browser-based, with no dependency on the operating system (OS)
2. Direct control of robot hardware via USB dongle using Web Serial communication
3. Support for simultaneous control of multiple robots – no limitation on robot type or quantity
4. When saving files, outputs are converted and stored as JSON text files

Real-Time Robot Control Method

In the robot coding programs provided by RobomationLAB, robots are controlled in real time through the following process.

1. Through block coding or script coding,
write code that sets values for Effector and Command objects to control the robot,
or utilizes the robot's Sensor values and Event occurrences.
2. Execute the code.
3. Receive packets containing Sensor and Event data from the robot via Web Serial communication and reflect them in the robot Device objects.
4. Interpret the code in real time,
overwrite data in Effector and Command objects, or read values from Sensor and Event objects.
5. Generate packets reflecting the data of the robot Device objects,

then send them to the robot via Web Serial communication to verify actual robot operation.

6. While the code is running, steps 3 to 5 are repeated approximately every 10–20 ms.

RobomationLAB - Robot Programming Method

Sequential Execution and Parallel Execution

There are two ways to program robots: sequential execution and parallel execution.

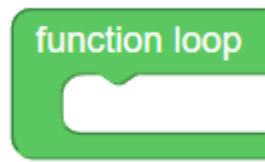
Sequential execution is a method in which the next action is performed after one action is completed, and it is suitable for coding simple behaviors.

For example, if you want the robot to move forward, then stop and turn on an LED, sequential execution is required so that the code corresponding to each action is arranged in order and executed chronologically.

Parallel execution is a method in which multiple actions are performed simultaneously, and it is necessary for programming more complex and advanced behaviors.

For example, to implement a walking motion for a bipedal robot, the robot's feet and legs must move at the same time, so parallel execution coding must be possible.

The robot coding programs provided by RobomationLAB support both sequential execution and parallel execution at the same time, based on a setup / loop code structure similar to Arduino's H/W development environment.



When you first access Block Composer, two empty function blocks appear in the workspace as shown above.

The "Start" block represents the setup function, and the "Repeat Forever" block represents the loop function.

Blocks are converted into Python / JavaScript code in real time, and the code has the following basic structure depending on the programming language.

```
# Python basic code structure
import asyncio
```

```
# put setup code here, to run once:
async def setup():
    return

# put control code here, to run repeatedly:
async def loop():
    return
```

```
// JavaScript basic code structure
// put setup code here, to run once:
async function setup() {
}

// put control code here, to run repeatedly:
async function loop() {
}
```

setup Function

The setup function is executed only once at the moment when “Run Code” is triggered.

In the setup function, code is mainly written to initialize variables or to initialize the robot’s mode, functions, and settings.

For example, when controlling a robot that moves using wheels, the initial wheel speed can be set in the setup function.

In addition, the `async` keyword attached before the setup function allows the code inside the function to be executed sequentially.

The `async/await` approach is a method added to modern JavaScript and Python, which designates a specific function as an asynchronous function and waits until its operation is completed without affecting the execution of other code.

In practice, the setup function is executed only once at the beginning of code execution.

However, if there is code with the `await` keyword inside the function, the function wakes up after a specified time or action and then executes the next code, so it can be considered to remain active.

By using this feature, not only simple sequential execution but also powerful robot programming is

possible through integration with the loop function, which plays the role of parallel execution.

The following is an example of writing code in which a HamsterS robot moves forward for 1 second and then moves backward for 1 second.

If this behavior were implemented inside the loop function using a parallel execution method, time calculations and control code would be mixed, making the code very complex.

Instead, by using the time-delay function `__wait` with the `await` keyword inside the setup function, it is possible to write code that operates in chronological order as if it were synchronous.

(The `__wait` function will be explained again later in Basic Utility Functions.)

Example Code (JavaScript)

```
// put setup code here, to run once:
async function setup() {
    // Move forward by setting both wheel speeds to 50
    $('HamsterS*0:wheel.speed.left').d = 50;
    $('HamsterS*0:wheel.speed.right').d = 50;
    await __wait(1000); // Wait for 1 second (1000 milliseconds)
    // Move backward by setting both wheel speeds to -50
    $('HamsterS*0:wheel.speed.left').d = -50;
    $('HamsterS*0:wheel.speed.right').d = -50;
    await __wait(1000); // Wait for 1 second (1000 milliseconds)
}

// put control code here, to run repeatedly:
function loop() {
}
```

(The \$ syntax will be explained again later in RobomationLAB - Basic Grammar System for Robot Coding.
)

loop Function

The loop function supports parallel execution and is repeatedly executed approximately every 10–20 ms while the code is running.

In the loop function, you typically write code that repeatedly updates variable values or detects and handles specific robot events.

Below is an example that changes the wheel speed and LED brightness of the HamsterS robot over time.

```
var frame;

// put setup code here, to run once:
async function setup() {
    frame = 0;
}

// put control code here, to run repeatedly:
function loop() {
    frame += 1; // Increase the value of the frame variable by 1 each time the
loop function is called

    // Use the updated frame value to set both wheel speeds and the RGB values of
both LEDs
    $('HamsterS*0:wheel.speed.left').d = frame % 100;
    $('HamsterS*0:wheel.speed.right').d = frame % 100;
    $('HamsterS*0:led.left').d = [frame % 256, 0, 0];
    $('HamsterS*0:led.right').d = [0, 0, frame % 256];
}
```

(The \$ syntax will be explained again later in RobomationLAB - Basic Grammar System for Robot Coding.
)

The following is an example of writing code that turns the LED red when a Tap action—lightly tapping the body of the HamsterS robot—occurs.

```
import asyncio

# put setup code here, to run once:
async def setup():
    return

# put control code here, to run repeatedly:
def loop():
    # Detect the moment when a Tap action occurs
```

```

if __('HamsterS*0:gravity.tap').e == True: # True when an event is detected
    __('HamsterS*0:led.left').d = [255, 0, 0] # Set the LED red value to 255
else:
    __('HamsterS*0:led.left').d = [0, 0, 0]
return

```

(The __ and .e syntax will be explained again later in RobomationLAB - Basic Grammar System for Robot Coding.)

Robot Software Modeling

This section defines the minimum system configuration for streaming-based robot hardware control and a JSON-based markup grammar.

Modeling Process

The robot software modeling process is broadly as follows. 1. Abstract and organize the functions, sensors, and characteristics of the robot hardware. 2. Using predefined blocks with specified classes, names, and data types, define Device objects that correspond one-to-one with the robot hardware. 3. Combine each defined Device object block to create a single model.

The model obtained by abstracting robot hardware into software in this way is called a Roboid. 4. Each block inside the Roboid is converted into a JSON data object according to predefined rules. 5. Subclass objects are connected in a structure that inherits the characteristics of superclass objects, and are ultimately converted into a single JSON dataset with a tree structure.

If block B is contained within block A, block A becomes the upper (parent) class object of block B, block B becomes the lower (child) class object of block A, and inherits the characteristics of block A.

Roboid Class Definition

The classes of Device objects that make up a Roboid are as follows.



Roboid

The Roboid class is the top-level parent class and represents which Roboid (robot) it is.

The Roboid class consists of one or more units or Devices as its components.

It has the properties name, implement (model name), and pid (unique product ID).

The delimiter used is :.



Module

The Module class is used to define modules that extend the functionality of the Roboid class.

External communication depends on the Roboid class, but functionally it is independent and internally has its own interface.

It has the same internal structure as the Roboid class and consists of one or more units or Devices as components.

For example, among the peripherals of the Raccoon robot arm, the Conveyor has its own internal interface, but external communication operates by depending on the Raccoon.

It has the properties name and mid (module unique ID).

The delimiter used is +.



Slot

The Slot class represents an interface to which a Module can be attached.

When selecting and using one module among multiple available Modules, the Slot can be used to distinguish them.

It has the properties name and module (connected module ID).

The delimiter used is >.



Unit

The Unit class is used simply to distinguish names without having any special functionality, and it serves as a container that groups Devices with similar characteristics.

It has the property name.

The delimiter used is ..

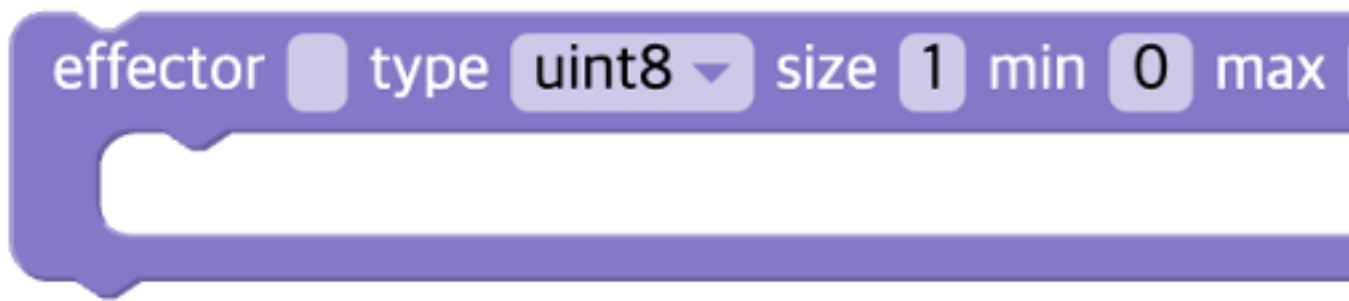
Notifier

A Notifier represents a type of device class that transmits information in a one-way manner at regular intervals, regardless of whether data reception is successful.

A device is classified as a Notifier if it meets any of the following conditions.

- 1) A device that changes rapidly and continuously over time
- 2) A device for which intermediate numerical values are meaningful
- 3) A device for which reception success is not important
- 4) A device for which information about execution progress or completion status is not required

Device objects that fall under the Notifier category include the Effector and Sensor classes.



Effector

The Effector class represents controllable devices among the physical components that make up a robot. Examples include a motor's speed value, an LED's brightness value, and a buzzer's frequency value. It has the properties name, type (data type), size (data size), min (minimum value), max (maximum value), and unit.

There is no delimiter.



Sensor

The Sensor class represents devices among the physical components of a robot that generate data. Examples include temperature sensors, acceleration sensors, and encoder values. It has the properties name, type (data type), size (data size), min (minimum value), max (maximum value), and unit.

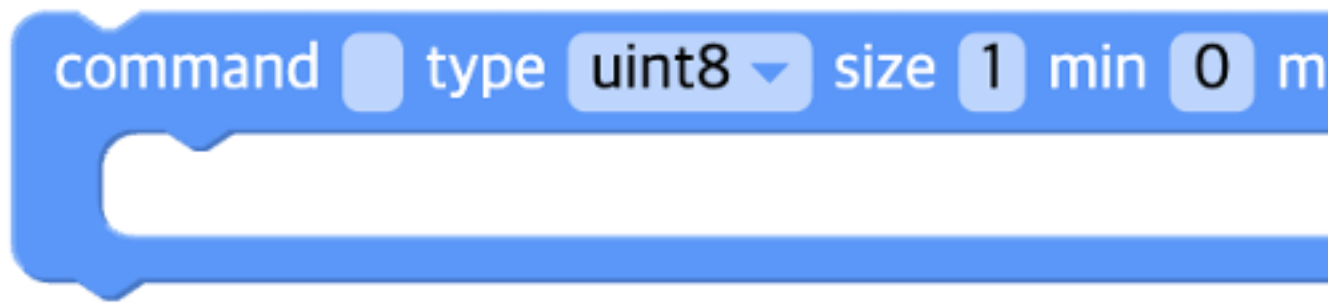
There is no delimiter.

Responder

Responder represents a device class for which it is necessary to determine whether data reception was successful.

A device is classified as a Responder if it satisfies at least one of the following conditions: 1) Devices where successful reception or completion status is important
2) Devices where intermediate values or execution process information are not required

Device objects that fall under the Responder category include the Command and Event classes.



Command

The Command class represents a controllable device similar to an Effector, and is used to define control commands that must be delivered to the robot.

By writing data to a Command object, control commands can be sent to the robot. Each time a command is sent, the internal `command_id` value is incremented, allowing the number of issued commands to be tracked.

It has the following properties:

name (name), type (data type), size (data size), min (minimum value), max (maximum value).

There is no separator.



Event

The Event class represents a device that generates data, similar to a Sensor, and is used to define specific events that must be detected.

Each time a response for a specific event is received from the robot, the internal `event_id` value is incremented, and at that moment the `e` value is set to `true`, allowing the occurrence of the event to be identified.

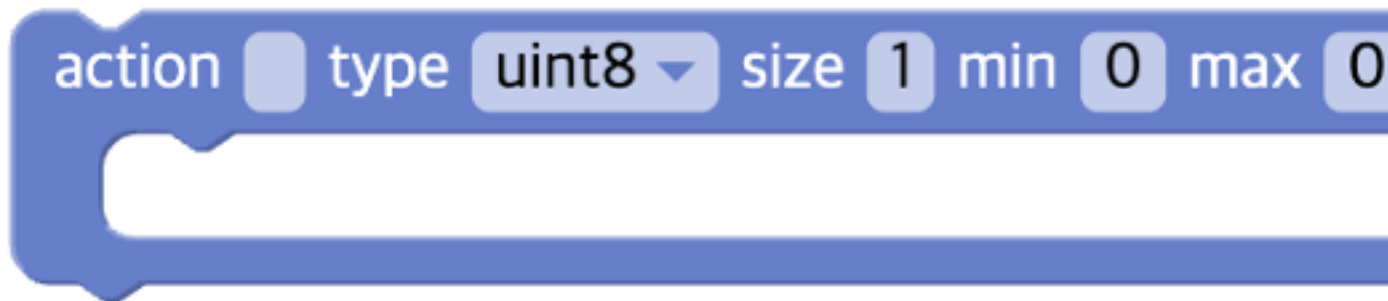
It has the following properties:

name (name), type (data type), size (data size), min (minimum value), max (maximum value).

There is no separator.

Action, Servo

The Action and Servo classes represent devices that require time to complete the execution of a command.



Action

The Action class is used when issuing a command for the robot to perform a specific action and when it is necessary to check whether that action has been completed.

It has the following properties:

name (name), type (data type), size (data size), min (minimum value), max (maximum value).

When an Action object is created, an Event object is automatically created internally.

The properties of the generated Event object are as follows:

Name: !{name} / Data type: uint8 / Data size: 0 / Minimum value: 0 / Maximum value: 0.

There is no separator.



Servo

The Servo class is used to issue commands for the robot to perform a specific action, to monitor the execution process, and to check whether the action has been completed.

Examples include setting robot joint angles and commands related to moving a certain distance or rotating by a certain angle.

It has the following properties:

name (name), type (data type), size (data size), min (minimum value), max (maximum value), unit (unit).

When a Servo object is created, Event and Sensor objects are automatically created internally.

The properties of the generated Event object are as follows:

Name: !{name} / Data type: uint8 / Data size: 0 / Minimum value: 0 / Maximum value: 0.

The generated Sensor object outputs intermediate values and is used to monitor the execution process.

Its name is set to ~{name}, and it has the same data type and size properties as the Servo object.

There is no separator.

####

Con-

stant

The
Con-
stant
class
is
used
to
pre-
de-
fine
con-
stant
val-
ues
that
each
De-
vice
ob-
ject
can
have
by de-
fault.
It has
the
fol-
low-
ing
prop-
er-
ties:
name
(name),
value
(value).

A
caret
sym-
bol ^
is
pre-
fixed
to
the
ob-
ject's
name.
There
is no
sepa-
rator.

Unique Identifier, URN

Each Roboid and Device object has a unique identifier called a URN.

The URN of each object is determined based on the following rules:

{ URN of a Roboid class object }

= { implement } + '*' + { index number } + { roboid object separator }

{ URN of a Device object }

= { URN of the parent Device object } + { name of the Device object } + { Device object separator }
{ }

When only one robot is used, the index number is 0.

When using multiple robots, the index number increases according to the number of robots of the same type

(e.g. 0, 1, 2, ...).

RobomationLAB - Basic Grammar System for Robot Coding

When writing code in the robot coding programs provided by RobomationLAB, the following basic syntax rules must be followed.

Robot Device Objects

Robot Device objects can be accessed in the following ways depending on the programming language.

```
# Python
__('{Device object URN}')
```

```
// JavaScript
$('{Device object URN}')
```

Using this object, you can write code to read or write values.

d Function

The d function is used to handle the data (value) of a Device object, allowing you to read from or write to the object.

It is mainly used to set values of Effector class objects, or to input commands into Command and Action class objects.

Depending on the programming language, the code is written as follows.

1. Reading a value

```
# Python
__('{Device object urn}').d
```

```
// JavaScript
$('{Device object urn}').d
```

2. Writing a value

```
# Python
__('{Device object urn}').d = {value}
```

```
// JavaScript
$('{Device object urn}').d = {value}
```

c Function

The c function is used to handle the command invocation count of a Device object.

It is read-only and returns the number of times a value has been written to the Device object.

It is mainly used to check how many times a specific action has been executed through a Command class object.

Depending on the programming language, the code is written as follows:

```
# Python
__('{Device object urn}').c
```

```
// JavaScript
$('{Device object urn}').c
```

e Function

The e function is used to handle event processing of a Device object.

It is read-only and is available only for Event class objects.

When a state change or an environmental change occurs in an Event object, this function detects the event and returns true at the moment the event occurs.

The e value of all Device objects is set before the loop function starts, and it is automatically reset to false after each execution of the loop function.

Depending on the programming language, the code is written as follows:

```
# Python
__('{Device object urn}').e
```

```
// JavaScript
$('{Device object urn}').e
```

w Function

The w function is used to wait for a Device object's operation-completion event. It is available only for Event class objects derived from Action or Servo classes, and it waits until the event indicating that a specific operation has been completed occurs.

After the event is detected, the program continues executing the next code.

Depending on the programming language, the code is written as follows:

```
# Python
await __('{Device object urn}').w()
```

```
// JavaScript
await $('{Device object urn}').w();
```

Coding-Rule

This document explains RobomationLAB, a robot coding education platform that is part of Robomation's "Execution Engine and Integrated Authoring Environment for Robot Motion Streaming Services".

It provides a detailed explanation in the following order:

- Main features of RobomationLAB - Robot programming methods of RobomationLAB - Principles of robot software modeling - Basic grammar system of RobomationLAB

Table of Contents

1. RobomationLAB - Robot Coding Education Platform
 - 1-1) Robot Coding Program
 - 1-2) Main Features of the Coding Program
 - 1-3) Real-Time Robot Control Method
2. RobomationLAB - Robot Programming Method
 - 2-1) Sequential Execution and Parallel Execution
 - 2-2) setup Function
 - 2-3) loop Function
3. Robot Software Modeling
 - 3-1) Modeling Process

- 3-2) Roboid Class Definition
- 3-3) Unique Identifier, urn
- 4. RobomationLAB - Basic Grammar System for Robot Coding
 - 4-1) Robot Device Objects
 - 4-2) d Function
 - 4-3) c Function
 - 4-4) e Function
 - 4-5) w Function

RobomationLAB - Robot Coding Education Platform

RobomationLAB is a platform optimized for **AI robotics software education**, providing a Chrome web browser-based integrated authoring environment for robot coding education targeting elementary and middle school students.

RobomationLAB offers various robot coding education programs such as **block coding** and **script coding**.

In addition to learning coding theory, learners can actually move and control robots using Robomation's products, providing opportunities to learn both coding and robotics at the same time.

Robot Coding Program

Block Composer

Block Composer is a tool that allows users to easily and quickly control Robomation robots through block coding and learn the basics of robot control.

- An authoring environment optimized for physical computing
- Easy coding for beginners using block Drag & Drop
- Learning environment that provides basic concepts and prevents syntax errors
- Automatic conversion to JavaScript and Python script code

- A collection of predefined blocks with robot-specific functions and various hands-on examples
- Real-time result verification through code execution
- Improvement of problem-solving skills and creativity through block combinations
- Optimized feedback provided through AI-based script code analysis

Script Composer

Script Composer is a tool that allows users to easily and quickly control Robomation robots through script coding and learn the basics of Python, JavaScript syntax, and robot coding.

- Simultaneous support for Python and JavaScript editors
- Automatic code completion and code insertion features by language
- Various hands-on example codes provided for each robot
- Real-time result verification through code execution
- Optimized feedback provided through AI-based script code analysis

Main Features of the Coding Program

The main features of the robot coding programs provided by RobomationLAB are as follows.

1. Chrome web browser-based, with no dependency on the operating system (OS)
2. Direct control of robot hardware via USB dongle using Web Serial communication
3. Support for simultaneous control of multiple robots – no limitation on robot type or quantity
4. When saving files, outputs are converted and stored as JSON text files

Real-Time Robot Control Method

In the robot coding programs provided by RobomationLAB, robots are controlled in real time through the following process.

1. Through block coding or script coding,

write code that sets values for Effector and Command objects to control the robot, or utilizes the robot's Sensor values and Event occurrences.

2. Execute the code.
3. Receive packets containing Sensor and Event data from the robot via Web Serial communication and reflect them in the robot Device objects.
4. Interpret the code in real time, overwrite data in Effector and Command objects, or read values from Sensor and Event objects.
5. Generate packets reflecting the data of the robot Device objects, then send them to the robot via Web Serial communication to verify actual robot operation.
6. While the code is running, steps 3 to 5 are repeated approximately every 10–20 ms.

RobomationLAB - Robot Programming Method

Sequential Execution and Parallel Execution

There are two ways to program robots: sequential execution and parallel execution.

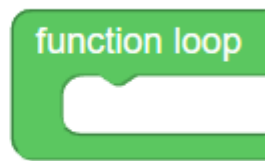
Sequential execution is a method in which the next action is performed after one action is completed, and it is suitable for coding simple behaviors.

For example, if you want the robot to move forward, then stop and turn on an LED, sequential execution is required so that the code corresponding to each action is arranged in order and executed chronologically.

Parallel execution is a method in which multiple actions are performed simultaneously, and it is necessary for programming more complex and advanced behaviors.

For example, to implement a walking motion for a bipedal robot, the robot's feet and legs must move at the same time, so parallel execution coding must be possible.

The robot coding programs provided by RobomationLAB support both sequential execution and parallel execution at the same time, based on a setup / loop code structure similar to Arduino's H/W development environment.



When you first access Block Composer, two empty function blocks appear in the workspace as shown above.

The “Start” block represents the setup function, and the “Repeat Forever” block represents the loop function.

Blocks are converted into Python / JavaScript code in real time, and the code has the following basic structure depending on the programming language.

```
# Python basic code structure
```

```
import asyncio
```

```
# put setup code here, to run once:
```

```
async def setup():  
    return
```

```
# put control code here, to run repeatedly:
```

```
async def loop():  
    return
```

```
// JavaScript basic code structure
```

```
// put setup code here, to run once:
```

```
async function setup() {  
}
```

```
// put control code here, to run repeatedly:
```

```
async function loop() {  
}
```

setup Function

The setup function is executed only once at the moment when “Run Code” is triggered.

In the setup function, code is mainly written to initialize variables or to initialize the robot’s mode, functions, and settings.

For example, when controlling a robot that moves using wheels, the initial wheel speed can be set in the setup function.

In addition, the async keyword attached before the setup function allows the code inside the function to be executed sequentially.

The async/await approach is a method added to modern JavaScript and Python,

which designates a specific function as an asynchronous function and waits until its operation is completed without affecting the execution of other code.

In practice, the setup function is executed only once at the beginning of code execution.

However, if there is code with the `await` keyword inside the function, the function wakes up after a specified time or action and then executes the next code,

so it can be considered to remain active.

By using this feature, not only simple sequential execution but also powerful robot programming is possible through integration with the loop function, which plays the role of parallel execution.

The following is an example of writing code in which a HamsterS robot moves forward for 1 second and then moves backward for 1 second.

If this behavior were implemented inside the loop function using a parallel execution method, time calculations and control code would be mixed, making the code very complex.

Instead, by using the time-delay function `__wait` with the `await` keyword inside the setup function, it is possible to write code that operates in chronological order as if it were synchronous.

(The `__wait` function will be explained again later in Basic Utility Functions.)

Example Code (JavaScript)

```
// put setup code here, to run once:
async function setup() {
    // Move forward by setting both wheel speeds to 50
    $('HamsterS*0:wheel.speed.left').d = 50;
    $('HamsterS*0:wheel.speed.right').d = 50;
    await __wait(1000); // Wait for 1 second (1000 milliseconds)
    // Move backward by setting both wheel speeds to -50
    $('HamsterS*0:wheel.speed.left').d = -50;
    $('HamsterS*0:wheel.speed.right').d = -50;
    await __wait(1000); // Wait for 1 second (1000 milliseconds)
}

// put control code here, to run repeatedly:
function loop() {
}
```

(The `$` syntax will be explained again later in RobomationLAB - Basic Grammar System for Robot Coding.

)

loop Function

The loop function supports parallel execution and is repeatedly executed approximately every 10–20 ms while the code is running.

In the loop function, you typically write code that repeatedly updates variable values or detects and handles specific robot events.

Below is an example that changes the wheel speed and LED brightness of the HamsterS robot over time.

```
var frame;

// put setup code here, to run once:
async function setup() {
  frame = 0;
}

// put control code here, to run repeatedly:
function loop() {
  frame += 1; // Increase the value of the frame variable by 1 each time the
loop function is called

  // Use the updated frame value to set both wheel speeds and the RGB values of
both LEDs
  $('HamsterS*0:wheel.speed.left').d = frame % 100;
  $('HamsterS*0:wheel.speed.right').d = frame % 100;
  $('HamsterS*0:led.left').d = [frame % 256, 0, 0];
  $('HamsterS*0:led.right').d = [0, 0, frame % 256];
}
```

(The \$ syntax will be explained again later in RobomationLAB - Basic Grammar System for Robot Coding.

)

The following is an example of writing code that turns the LED red when a Tap action—lightly tapping the body of the HamsterS robot—occurs.

```
import asyncio
```

```

# put setup code here, to run once:
async def setup():
    return

# put control code here, to run repeatedly:
def loop():
    # Detect the moment when a Tap action occurs
    if __('HamsterS*0:gravity.tap').e == True: # True when an event is detected
        __('HamsterS*0:led.left').d = [255, 0, 0] # Set the LED red value to 255
    else:
        __('HamsterS*0:led.left').d = [0, 0, 0]
    return

```

(The `__` and `.e` syntax will be explained again later in RobomationLAB - Basic Grammar System for Robot Coding.)

Robot Software Modeling

This section defines the minimum system configuration for streaming-based robot hardware control and a JSON-based markup grammar.

Modeling Process

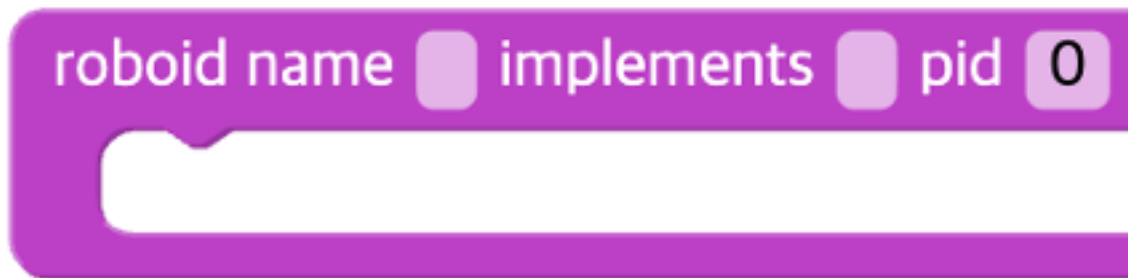
The robot software modeling process is broadly as follows. 1. Abstract and organize the functions, sensors, and characteristics of the robot hardware. 2. Using predefined blocks with specified classes, names, and data types, define Device objects that correspond one-to-one with the robot hardware. 3. Combine each defined Device object block to create a single model.

The model obtained by abstracting robot hardware into software in this way is called a Roboid. 4. Each block inside the Roboid is converted into a JSON data object according to predefined rules. 5. Subclass objects are connected in a structure that inherits the characteristics of superclass objects, and are ultimately converted into a single JSON dataset with a tree structure.

If block B is contained within block A, block A becomes the upper (parent) class object of block B, block B becomes the lower (child) class object of block A, and inherits the characteristics of block A.

Roboid Class Definition

The classes of Device objects that make up a Roboid are as follows.



Roboid

The Roboid class is the top-level parent class and represents which Roboid (robot) it is.

The Roboid class consists of one or more units or Devices as its components.

It has the properties name, implement (model name), and pid (unique product ID).

The delimiter used is :.



Module

The Module class is used to define modules that extend the functionality of the Roboid class.

External communication depends on the Roboid class, but functionally it is independent and internally has its own interface.

It has the same internal structure as the Roboid class and consists of one or more units or Devices as components.

For example, among the peripherals of the Raccoon robot arm, the Conveyor has its own internal interface, but external communication operates by depending on the Raccoon.

It has the properties name and mid (module unique ID).

The delimiter used is +.



Slot

The Slot class represents an interface to which a Module can be attached.

When selecting and using one module among multiple available Modules, the Slot can be used to distinguish them.

It has the properties name and module (connected module ID).

The delimiter used is >.



Unit

The Unit class is used simply to distinguish names without having any special functionality, and it serves as a container that groups Devices with similar characteristics.

It has the property name.

The delimiter used is ..

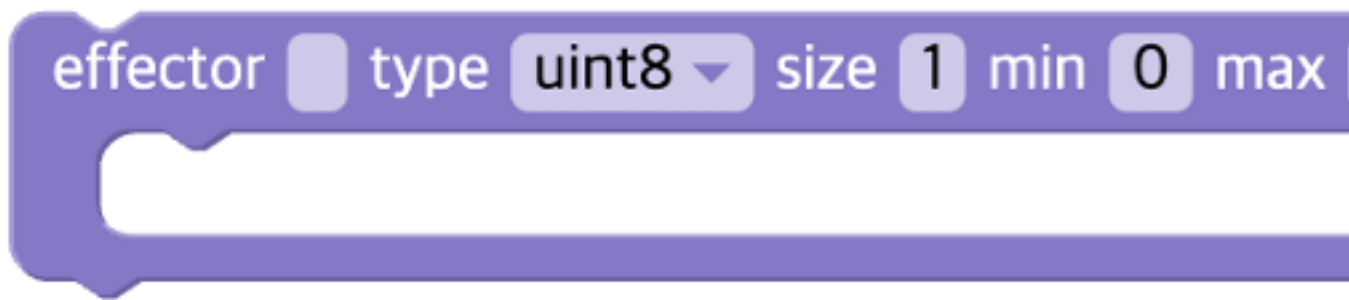
Notifier

A Notifier represents a type of device class that transmits information in a one-way manner at regular intervals, regardless of whether data reception is successful.

A device is classified as a Notifier if it meets any of the following conditions.

- 1) A device that changes rapidly and continuously over time
- 2) A device for which intermediate numerical values are meaningful
- 3) A device for which reception success is not important
- 4) A device for which information about execution progress or completion status is not required

Device objects that fall under the Notifier category include the Effector and Sensor classes.



Effector

The Effector class represents controllable devices among the physical components that make up a robot. Examples include a motor's speed value, an LED's brightness value, and a buzzer's frequency value. It has the properties name, type (data type), size (data size), min (minimum value), max (maximum value), and unit.

There is no delimiter.



Sensor

The Sensor class represents devices among the physical components of a robot that generate data. Examples include temperature sensors, acceleration sensors, and encoder values. It has the properties name, type (data type), size (data size), min (minimum value), max (maximum value), and unit.

There is no delimiter.

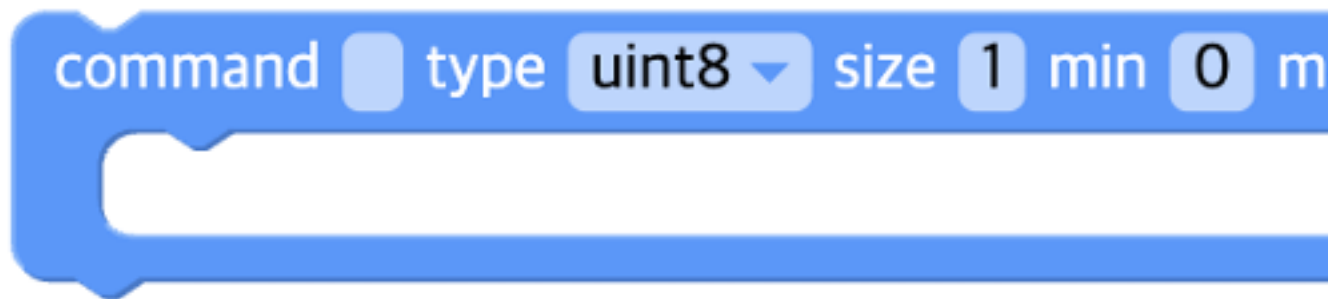
Responder

Responder represents a device class for which it is necessary to determine whether data reception was successful.

A device is classified as a Responder if it satisfies at least one of the following conditions: 1) Devices where successful reception or completion status is important

2) Devices where intermediate values or execution process information are not required

Device objects that fall under the Responder category include the Command and Event classes.



Command

The Command class represents a controllable device similar to an Effector, and is used to define control commands that must be delivered to the robot.

By writing data to a Command object, control commands can be sent to the robot. Each time a command is sent, the internal `command_id` value is incremented, allowing the number of issued commands to be tracked.

It has the following properties:

name (name), type (data type), size (data size), min (minimum value), max (maximum value).

There is no separator.



Event

The Event class represents a device that generates data, similar to a Sensor, and is used to define specific events that must be detected.

Each time a response for a specific event is received from the robot, the internal `event_id` value is incremented, and at that moment the `e` value is set to `true`, allowing the occurrence of the event to be identified.

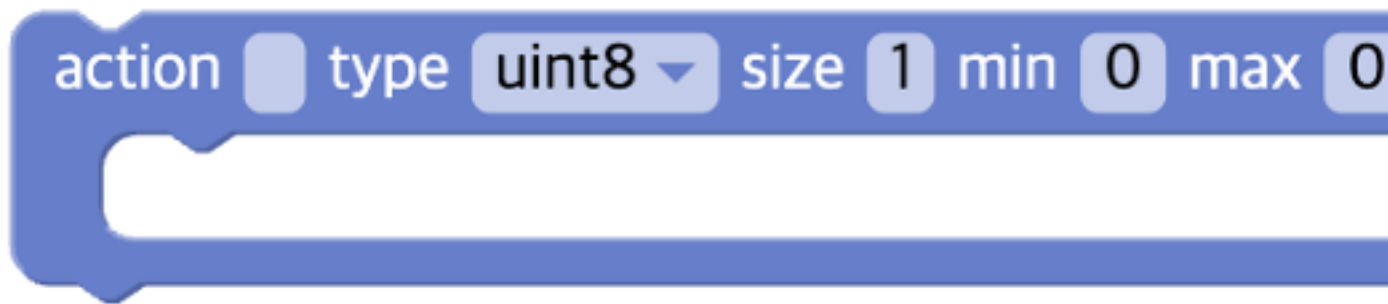
It has the following properties:

name (name), type (data type), size (data size), min (minimum value), max (maximum value).

There is no separator.

Action, Servo

The Action and Servo classes represent devices that require time to complete the execution of a command.



Action

The Action class is used when issuing a command for the robot to perform a specific action and when it is necessary to check whether that action has been completed.

It has the following properties:

name (name), type (data type), size (data size), min (minimum value), max (maximum value).

When an Action object is created, an Event object is automatically created internally.

The properties of the generated Event object are as follows:

Name: !{name} / Data type: uint8 / Data size: 0 / Minimum value: 0 / Maximum value: 0.

There is no separator.



Servo

The Servo class is used to issue commands for the robot to perform a specific action, to monitor the execution process, and to check whether the action has been completed.

Examples include setting robot joint angles and commands related to moving a certain distance or rotating by a certain angle.

It has the following properties:

name (name), type (data type), size (data size), min (minimum value), max (maximum value), unit (unit).

When a Servo object is created, Event and Sensor objects are automatically created internally.

The properties of the generated Event object are as follows:

Name: !{name} / Data type: uint8 / Data size: 0 / Minimum value: 0 / Maximum value: 0.

The generated Sensor object outputs intermediate values and is used to monitor the execution process.

Its name is set to ~{name}, and it has the same data type and size properties as the Servo object.

There is no separator.

Con-
stant

The
Con-
stant
class
is
used
to
pre-
de-
fine
con-
stant
val-
ues
that
each
De-
vice
ob-
ject
can
have
by de-
fault.
It has
the
fol-
low-
ing
prop-
er-
ties:
name
(name),
value
(value).

A
caret
sym-
bol ^
is
pre-
fixed
to
the
ob-
ject's
name.
There
is no
sepa-
rator.

Unique Identifier, URN

Each Roboid and Device object has a unique identifier called a URN.

The URN of each object is determined based on the following rules:

{ URN of a Roboid class object }

= { implement } + '*' + { index number } + { roboid object separator }

{ URN of a Device object }

= { URN of the parent Device object } + { name of the Device object } + { Device object separator }
{ }

When only one robot is used, the index number is 0.

When using multiple robots, the index number increases according to the number of robots of the same type

(e.g. 0, 1, 2, ...).

RobomationLAB - Basic Grammar System for Robot Coding

When writing code in the robot coding programs provided by RobomationLAB, the following basic syntax rules must be followed.

Robot Device Objects

Robot Device objects can be accessed in the following ways depending on the programming language.

```
# Python
__('{Device object URN}')
```

```
// JavaScript
$('{Device object URN}')
```

Using this object, you can write code to read or write values.

d Function

The d function is used to handle the data (value) of a Device object, allowing you to read from or write to the object.

It is mainly used to set values of Effector class objects, or to input commands into Command and Action class objects.

Depending on the programming language, the code is written as follows.

1. Reading a value

```
# Python
__('{Device object urn}').d
```

```
// JavaScript
$('{Device object urn}').d
```

2. Writing a value

```
# Python
__('{Device object urn}').d = {value}
```

```
// JavaScript
$('{Device object urn}').d = {value}
```

c Function

The c function is used to handle the command invocation count of a Device object.

It is read-only and returns the number of times a value has been written to the Device object.

It is mainly used to check how many times a specific action has been executed through a Command class object.

Depending on the programming language, the code is written as follows:

```
# Python
__('{Device object urn}').c
```

```
// JavaScript
$('{Device object urn}').c
```

e Function

The e function is used to handle event processing of a Device object.

It is read-only and is available only for Event class objects.

When a state change or an environmental change occurs in an Event object, this function detects the event and returns true at the moment the event occurs.

The e value of all Device objects is set before the loop function starts, and it is automatically reset to false after each execution of the loop function.

Depending on the programming language, the code is written as follows:

```
# Python
__('{Device object urn}').e
```

```
// JavaScript
$('{Device object urn}').e
```

w Function

The w function is used to wait for a Device object's operation-completion event. It is available only for Event class objects derived from Action or Servo classes, and it waits until the event indicating that a specific operation has been completed occurs.

After the event is detected, the program continues executing the next code.

Depending on the programming language, the code is written as follows:

```
# Python
await __('{Device object urn}').w()
```

```
// JavaScript
await $('{Device object urn}').w();
```

Custom-Function

This document explains custom functions that support smooth robot coding in RobomationLAB.

Basic Utility Functions

Basic utility functions are functions that can be commonly used in the robot coding programs provided by RobomationLAB.

Except for the Wait function, utility functions use the same code regardless of Python or JavaScript syntax.

Below, the role of each function and how to write the code are explained.

Wait

This is a time-delay function used when executing code in a sequential manner.

After waiting for the amount of time specified as a parameter, the next code is executed.

The await keyword must be placed in front of it, and it can only be used inside functions marked with async.

Parameters

Parameter	Range	Description
time	(Integer ≥ 0)	Time (Python: seconds, JavaScript: milliseconds)

The wait function is an exception and uses different code depending on Python or JavaScript syntax.

Python Code

```
await asyncio.sleep(1) # (seconds)
```

JavaScript Code

```
await __wait(1000); // (milliseconds)
```

Key Down

This function detects keyboard input and returns whether the keyCode provided as a parameter matches the last pressed key.

Parameters

Parameter	Range	Description
keyCode	(Integer ≥ 0)	keyCode value corresponding to a keyboard key

Code

```
__keydown(0) # (keyCode)
```

Key Up

This function detects keyboard input and returns whether the keyCode provided as a parameter matches the last released key.

Parameters

Parameter	Range	Description
keyCode	(Integer ≥ 0)	keyCode value corresponding to a keyboard key

Code

```
__keyup(0) # (keyCode)
```

Key Pressed

This function detects keyboard input and returns whether the set of currently pressed keys matches the given keyCode array provided as a parameter.

Parameters

Parameter	Range	Description
keyCode list	(Array)	List of keyCode values corresponding to keyboard keys

Code

```
__keypressed([0, 0, 0, 0, 0]) # (keyCode list)
```

Log

Console - This is a function that outputs the value of a specific variable to the log.

Parameters

Parameter	Range	Description
signal	-	Value to be output to the log
tag	(String)	Tag (name) to be displayed in the log
unit	(String)	Unit to be displayed in the log

Parameter	Range	Description
-----------	-------	-------------

Code

```
__log(0, '', '') # (signal, title, unit)
```

Scope

Console - This is a function that outputs the changes in specific variable values as a graph in the Scope window.

Parameters

Parameter	Range	Description
title	(String)	Scope title
min	(Integer)	Minimum value
max	(Integer)	Maximum value
color	(String)	Line graph color
signal	-	Value to be displayed in the scope

Code

```
__scope('', 0, 1, '#000000', 0); # (title, min, max, color, signal)
```

Random Color

This function returns a [red, green, blue] array with random RGB values.

Code

```
__randomColor()
```

Play Sound

This function plays a sound registered in the program through the PC or device.

Parameters

Parameter	Range	Description
name	-	Name of the sound to play
volume	(Integer, 0 ~ 100)	Sound volume
repeat	(Boolean)	Whether to repeat the sound

Code

```
__playSound('', 100, True) # (name, volume, repeat)
```

Speak

This function converts text into speech and plays it through the PC or device.

Parameters

Parameter	Range	Description
text	(String)	Text to be played using TTS

Code

```
__speak('') # text
```

Exit

This function terminates code execution.

Code

```
__exit()
```

Robot-Specific Special Functions

Robot-specific special functions are dedicated functions provided according to the hardware characteristics of each robot.

They are used to perform actual actions or calculations such as robot movement, rotation, and kinematic operations.

Below, the role of each function, how to write the code, and the robots that use these functions are explained.

Get Speed

This function converts the robot wheel speed entered as a value between 0 and 100 into the value required for the robot to actually move.

Robots that use this function:

Hamster S, Hamster, Pio, Turtle, Beagle

Parameters

Parameter	Range	Description
robot	(String)	Name of the robot to control
value	(Float, 0 ~ 100)	Speed value to convert

Python Code

Python

```
__getSpeed(' ', 50) # (robot, value)
```

JavaScript Code

```
// JavaScript  
__getSpeed(' ', 50) // (robot, value)
```

Get Distance

This function converts the input distance (e.g., 30 cm, 500 mm) into the value required for the robot to actually move.

Robots that use this function:

Hamster S, Pio, Turtle, Beagle, RaccoonBot peripherals – Conveyor, Slider

Parameters

Parameter	Range	Description
robot	(String)	Name of the robot to control
value	(Float)	Distance value to convert
unit	('cm' or 'mm')	Distance unit

Python Code

```
__getDistance('', 50, 'cm') # (robot, value, unit)
```

JavaScript Code

```
__getDistance('', 50, 'cm') // (robot, value, unit)
```

Turn Left in Place

This function rotates the robot to the left by **n** degrees while staying in place.

Robots that use this function:

Hamster S, Pio, Turtle, Beagle

Parameters

Parameter	Range	Description
robot	(String)	Name of the robot to control

Parameter	Range	Description
degree	(Integer, 0 ~ 360)	Rotation angle
wait_w	(Boolean)	Whether to wait

If wait_w is set to true, use the await keyword to wait until the rotation is completed, then execute the next code.

If wait_w is set to false, the next code is executed immediately.

Python

```
await __turn_degree_left('', 90, True) # (robot, degree, wait_w)
__turn_degree_left('', 90, False) # (robot, degree, wait_w)
```

JavaScript

```
await __turn_degree_left('', 90, true); // (robot, degree, wait_w)
__turn_degree_left('', 90, false); // (robot, degree, wait_w)
```

Turn Right in Place

This function rotates the robot to the right by **n** degrees while staying in place.

Robots that use this function:

Hamster S, Pio, Turtle, Beagle

Parameters

Parameter	Range	Description
robot	(String)	Name of the robot to control
degree	(Integer, 0 ~ 360)	Rotation angle
wait_w	(Boolean)	Whether to wait

If wait_w is set to true, use the await keyword to wait until the rotation is completed, then stop and

execute the next code.

If `wait_w` is set to false, the next code is executed immediately.

Python Code

```
await __turn_degree_right('', 90, True) # (robot, degree, wait_w)
__turn_degree_right('', 90, False) # (robot, degree, wait_w)
```

JavaScript Code

```
await __turn_degree_right('', 90, true); // (robot, degree, wait_w)
__turn_degree_right('', 90, false); // (robot, degree, wait_w)
```

Turbo

This function enables or disables the robot's turbo mode.

Robots that use this function:

Pio

Parameters

Parameter	Range	Description
robot	(String)	Name of the robot to control
turbo	(Boolean)	Whether to enable turbo mode

Python

```
__turbo('', True) # (robot, turbo)
__turbo('', False) # (robot, turbo)
```

JavaScript

```
__turbo(' ', false); // (robot, turbo)
__turbo(' ', true); // (robot, turbo)
```

Stop Move

This function stops the robot's wheel movement.

Robots that use this function:

Hamster S (HamsterS), Pio, Turtle, Beagle

Parameters

Parameter	Range	Description
robot	(String)	Name of the robot to control

Python Code

```
__stopMove(' ') # (robot)
```

JavaScript Code

```
__stopMove(' '); // (robot)
```

Stop After Delay

This function stops the robot's wheel movement after waiting for a specified amount of time.

Robots that use this function:

Hamster S (HamsterS), Hamster, Pio, Turtle, Beagle

Parameters

Parameter	Range	Description
robot	(String)	Name of the robot to control
delay	(Float ≥ 0)	Waiting time before stopping
wait_w	(Boolean)	Whether to wait

If wait_w is set to true, use the await keyword to wait until the specified time has passed, then stop the wheel movement and execute the next code.

If wait_w is set to false, the next code is executed immediately, and the wheel movement stops after the specified time has passed.

Python

```
await __stopAfterDelay('', 5, True) # (robot, delay, wait_w)
__stopAfterDelay('', 90, False) # (robot, delay, wait_w)
```

JavaScript

```
await __stopAfterDelay('', 5, true); // (robot, delay, wait_w)
__stopAfterDelay('', 90, false); // (robot, delay, wait_w)
```

Grid Turn Left

This function rotates the robot one step to the left while it is on the grid board.

Robots that use this function:

Hamster S, Hamster, Pio

Parameters

Parameter	Range	Description
robot	(String)	Name of the robot to control
wait_w	(Boolean)	Whether to wait

If `wait_w` is set to `true`, use the `await` keyword to wait until the movement is completed, then stop and execute the next code.

If `wait_w` is set to `false`, the next code is executed immediately.

Python Code

```
await __grid_turn_left('', True)  # (robot, wait_w)
__grid_turn_left('', False)  # (robot, wait_w)
```

JavaScript Code

```
await __grid_turn_left('', true); // (robot, wait_w)
__grid_turn_left('', false); // (robot, wait_w)
```

Grid Turn Right

This function rotates the robot one step to the right while it is on the grid board.

Robots that use this function:

Hamster S, Hamster, Pio

Parameters

Parameter	Range	Description
robot	(String)	Name of the robot to control
wait_w	(Boolean)	Whether to wait

If `wait_w` is set to `true`, use the `await` keyword to wait until the movement is completed, then stop and execute the next code.

If `wait_w` is set to `false`, the next code is executed immediately.

Python Code

```
await __grid_turn_right('', True) # (robot, wait_w)
__grid_turn_right('', False) # (robot, wait_w)
```

JavaScript Code

```
await __grid_turn_right('', true); // (robot, wait_w)
__grid_turn_right('', false); // (robot, wait_w)
```

Grid Move Forward

This function moves the robot forward by one cell while it is on the grid board.

Robots that use this function:

Hamster S, Hamster, Pio

Parameters

Parameter	Range	Description
robot	(String)	Name of the robot to control
wait_w	(Boolean)	Whether to wait

If wait_w is set to true, use the await keyword to wait until the movement is completed, then stop and execute the next code.

If wait_w is set to false, the next code is executed immediately.

Python Code

```
await __grid_move_forward('', True) # (robot, wait_w)
__grid_move_forward('', False) # (robot, wait_w)
```

JavaScript Code

```
await __grid_move_forward('', true); // (robot, wait_w)
__grid_move_forward('', false); // (robot, wait_w)
```

Grid Move Backward

This function moves the robot backward by one cell while it is on the grid board.

Robots that use this function:

Pio

Parameters

Parameter	Range	Description
robot	(String)	Name of the robot to control
wait_w	(Boolean)	Whether to wait

If wait_w is set to true, use the await keyword to wait until the movement is completed, then stop and execute the next code.

If wait_w is set to false, the next code is executed immediately.

Python Code

```
await __grid_move_backward('', True) # (robot, wait_w)
__grid_move_backward('', False) # (robot, wait_w)
```

JavaScript Code

```
await __grid_move_backward('', true); // (robot, wait_w)
__grid_move_backward('', false); // (robot, wait_w)
```

Grid Move Left

This function moves the robot one cell to the left while it is on the grid board.

Robots that use this function:

Pio

Parameters

Parameter	Range	Description
robot	(String)	Name of the robot to control
wait_w	(Boolean)	Whether to wait

If `wait_w` is set to `true`, use the `await` keyword to wait until the movement is completed, then stop and execute the next code.

If `wait_w` is set to `false`, the next code is executed immediately.

Python Code

```
await __grid_move_left('', True) # (robot, wait_w)
__grid_move_left('', False) # (robot, wait_w)
```

JavaScript Code

```
await __grid_move_left('', true); // (robot, wait_w)
__grid_move_left('', false); // (robot, wait_w)
```

Grid Move Right

This function moves the robot one cell to the right while it is on the grid board.

Robots that use this function:

Pio

Parameters

Parameter	Range	Description
robot	(String)	Name of the robot to control
wait_w	(Boolean)	Whether to wait

If wait_w is set to true, use the await keyword to wait until the movement is completed, then stop and execute the next code.

If wait_w is set to false, the next code is executed immediately.

Python Code

```
await __grid_move_right('', True) # (robot, wait_w)
__grid_move_right('', False) # (robot, wait_w)
```

JavaScript Code

```
await __grid_move_right('', true); // (robot, wait_w)
__grid_move_right('', false); // (robot, wait_w)
```

Pivot

This function rotates the robot in place by **n** degrees in a desired direction based on a specific pivot axis.

Robots that use this function:

Hamster S, Turtle

Parameters

Parameter	Range	Description
robot	(String)	Name of the robot to control
unit	(String)	Pivot axis
direction	(String)	Rotation direction
degree	(Integer, 0 ~ 360)	Rotation angle
wait_w	(Boolean)	Whether to wait

If `wait_w` is set to `true`, use the `await` keyword to wait until the rotation is completed, then stop and execute the next code.

If `wait_w` is set to `false`, the next code is executed immediately.

Unit Options

The `unit` parameter can have the following values depending on the robot:

Hamster S (HamsterS): left pen, right pen, left wheel, right wheel

Turtle: left wheel, right wheel

Direction Options

The `direction` parameter can have the following values regardless of the robot:

forward, backward

Python Code

```
await __pivot('', '', '', 90, True) # (robot, unit, direction, degree, wait_w)
__pivot('', '', '', 90, False) # (robot, unit, direction, degree, wait_w)
```

JavaScript Code

```
await __pivot('', '', '', 90, true); // (robot, unit, direction, degree, wait_w)

__pivot('', '', '', 90, false); // (robot, unit, direction, degree, wait_w)
```

Pivot Circle

This function rotates the robot by **n** degrees in a desired direction while drawing a circle with radius **k** **cm** based on a specific pivot axis.

Robots that use this function:

Hamster S, Turtle

Parameters

Parameter	Range	Description
robot	(String)	Name of the robot to control
unit	(String)	Pivot axis
direction	(String)	Rotation direction
degree	(Integer, 0 ~ 360)	Rotation angle
wait_w	(Boolean)	Whether to wait

If wait_w is set to true, use the await keyword to wait until the rotation is completed, then stop and execute the next code.

If wait_w is set to false, the next code is executed immediately.

Unit Options

The unit parameter can have the following values depending on the robot:

Hamster S (HamsterS): left pen, right pen

Turtle: "

Direction Options

The direction parameter can have the following values regardless of the robot:

left forward, left backward, right forward, right backward

Python Code

```
await __pivot_circle('', '', '', __getDistance('', 1, 'cm'), 90, True) # (robot,
unit, direction, radius, degree, wait_w)
__pivot_circle('', '', '', __getDistance('', 1, 'cm'), 90, False) # (robot,
unit, direction, radius, degree, wait_w)
```

JavaScript Code

```
await __pivot_circle('', '', '', __getDistance('', 1, 'cm'), 90, true); //
(robot, unit, direction, radius, degree, wait_w)
__pivot_circle('', '', '', __getDistance('', 1, 'cm'), 90, false); // (robot,
unit, direction, radius, degree, wait_w)
```

Stop Sound

This function stops sound playback on the robot.

Robots that use this function:

Hamster S (HamsterS), Hamster, Turtle, Pio, Beagle, RaccoonBot (Raccoon4)

Python Code

```
__stopSound('') # (robot)
```

JavaScript Code

```
__stopSound(''); // (robot)
```

Forward Kinematics (angles to xyz)

This function calculates and returns the coordinates of the end-effector's current position based on the robot arm's joint angles.

It converts each joint angle into xyz coordinates using the Forward Kinematics algorithm.

Robots that use this function:

RaccoonBot (Raccoon4)

Parameters

Parameter	Range	Description
robot	(String)	Name of the robot to control
angles	(Array)	Joint angle array [j1, j2, j3, j4]
origin	(String)	Reference point for coordinate calculation ('wrist', 'end_effector')

Python Code

```
__angles_to_xyz('', [0, 0, 0, 0], 'wrist') # (robot, angles, origin)
```

JavaScript Code

```
__angles_to_xyz('', [0, 0, 0, 0], 'end_effector') // (robot, angles, origin)
```

Inverse Kinematics (xyz to angles)

This function calculates and returns the four joint angles of the robot arm based on the current coordinates of the end-effector.

It converts xyz coordinates into four joint angles using the Inverse Kinematics algorithm.

Robots that use this function:

RaccoonBot (Raccoon4)

Parameters

Parameter	Range	Description
robot	(String)	Name of the robot to control
xyz	(Array)	Coordinate array of the end-effector [x, y, z]
origin	(String)	Reference point for coordinate calculation ('wrist', 'end_effector')

JavaScript Code

```
__xyz_to_angles('', [0, 100, 100], 'wrist'); // (robot, xyz, origin)
```

Python Code

```
__xyz_to_angles('', [0, 100, 100], 'end_effector') # (robot, xyz, origin)
```